

The 64-Copy Problem

Participant Data Kit — read this before writing any code

You are given ground states of the spin-1/2 **XXZ chain** (16 qubits, periodic boundary conditions, with a tiny symmetry-breaking field $h = 1e-4$ along z). Your task: classify the **quantum phase** of unseen states — **FM**, **XY**, or **NEEL** — or flag a state as **UNKNOWN** if it does not belong to the XXZ family at all. Impostor states in the hidden set are drawn from a **1D Fermi-Hubbard model** (8 sites, Jordan-Wigner mapped to 16 qubits).

What's in this kit

File	Contents
xxz_public_train.npz	48 labeled training states: states (48×65536 float32), ids, deltas, labels
oracle.py	The copy-metered oracle — the ONLY way test states can be accessed
starter.py	Loading, oracle API demo, and the required submission interface

Conventions (also embedded in the npz): `qml.StatePrep(state, wires=range(16))` places chain site i on wire i ; $|0\rangle$ = spin up. Labels follow the thermodynamic phase boundaries: FM for $\Delta < -1$, XY for $-1 < \Delta < 1$, NEEL for $\Delta > 1$.

The rules of the game

- 1. Training states are yours.** Full state vectors, labels, Δ values — use them however you like, without limits.
- 2. Test states exist only behind the oracle.** For each hidden state you get a budget of **64 copies**. One call to `oracle.measure(...)` = one copy: the state is prepared fresh (through the noise channel), your gates run on it, one measurement sample comes back, the copy is destroyed.
- 3. Noise.** Every copy passes through per-qubit **depolarizing noise** before your operations. The probability used in final evaluation is not announced. The practice oracle lets you set any `noise_p` you want.
- 4. Budget violations forfeit that state.** The oracle raises `CopyBudgetExceeded`; catch it or count your copies.
- 5. Adaptivity is allowed.** You may decide what to do with copy k based on the outcomes of copies $1..k-1$.

Submission contract

Your repository must contain `submission.py` defining exactly:

```
def classify(oracle, state_ids):
    """Return {state_id: label}, label in
    {"FM", "XY", "NEEL", "UNKNOWN"},
    for every id in state_ids."""
```

Organizers will run exactly this function against the hidden set with the same `CopyOracle` class you have in this kit. Anything your protocol needs must happen inside this call (models may be pre-trained and loaded from files in your repo). Scoring per the track document: macro-F1 over the three phases, impostor precision/recall, per-tier breakdown, plus your report and analysis.

Practice loop: wrap the public states in the oracle (`starter.py`, section 4), self-score against the known labels, and log your copy usage — you will be asked about it in the report.